

Plan Recognition for Interface Agents

State of the art

Marcelo Gabriel Armentano (marmenta@exa.unicen.edu.ar) and
Analía Amandi (amandi@exa.unicen.edu.ar).
*ISISTAN Research Institute,
UNICEN University,
Campus Universitario (B7001BBO), Tandil, Buenos Aires, Argentina.
Also CONICET.*

Abstract. Interface agents are computer programs that provide personalized assistance to a user dealing with computer based applications. By understanding the tasks the user performs in a software application an interface agent could be aware of the context that represents the user's focus of attention at each particular moment. With this purpose, plan recognition aims at identifying the plans or goals of a user from the tasks he¹ performs. A prerequisite for the recognition of plans is knowledge of a user's possible tasks and the combination of these tasks in complex task sequences, which describes typical user behavior. Plan recognition will enable an interface agent to reason about what the user might do next so that it can determine how to assist him. In this work we present the state of the art in Plan Recognition, paying special attention to the features that make it useful to interface agents. These features include the ability to deal with uncertainty, multiple plans, multiple interleaved goals, overloaded tasks, noisy tasks, interruptions and the capability to adapt to a particular user.

Keywords: Plan Recognition, Interface Agents

1. Introduction

Consider the following situation. John, a personal assistant, sees his boss opening a personal information management software application that both of them know very well. This software application has many tools including managing events, contacts, reminders, home budget, personal diary, lists of favorite web pages, passwords, etc. The boss can use any of the tools the software application offers. Next, John observes his boss clicking on the "Add event" button. Now, John is sure his boss wants to add a new event to the calendar. John knows the steps required to complete that task, and he is able to help his boss at any moment. The boss enters the subject of the event, the day and starting time and then presses the "Add participants" button. After scrolling the list of contacts, he presses the "Cancel" button and saves the event. John cannot imagine what his boss intends to do. Perhaps his boss has

¹ For simplicity, we use "he" to refer to the user, but we do not mean any distinctions about sexes.



not found the contact he has been looking for, or may be he has a meeting and cannot spend more time scheduling the event. But next John observes his boss pressing the “Add contact” button, completing the required information and saving it. With the information John has so far, he can think that the participant his boss has intended to add was not in his contact list so he had to add it, and after that he will edit the event he had saved before and add the new contact to it as a participant. Moreover, if the personal assistant has enough knowledge about his boss habits and the events he usually adds to the application, he could be able to predict, for example, that the event will take place in a specific location, at an specific time just by observing the subject, the host and other information the boss has entered. It may also be possible for John to predict that some member of the office will take part of the meeting based on the type of meeting the boss is scheduling.

What the personal assistant has done by trying to deduce his boss intention is a *plan recognition* process. Specifically, plan recognition can be defined as the process of inferring the intentions of a subject based on the actions he performs in an environment. Plan recognition has been applied to different computer applications such as discourse analysis (Grosz and Sidner, 1990), collaborative planning (Huber and Durfee, 1993), adversarial planning (Azarewicz et al., 1989), story understanding (Charniak and Goldman, 1993; Albrecht et al., 1998), intelligent tutoring (Koehn and Greer, 1995), help systems (Breuker, 1990; Horvitz et al., 1998), interface and collaborative agents (Lesh, 1999; Rich and Sidner, 1997; Brown, 1998).

From the approaches mentioned above, our work is framed within interface agent technology. Interface agents are computer programs designed to assist users with their computer-based tasks in a personalized manner. Assistance is based on what the interface agent learns about the user’s interests, preferences, priorities, goals and needs, and is done by providing him proactive and reactive assistance in order to increase the user’s productivity with respect to a computer application. A commonly used metaphor to understand the interface agent paradigm is comparing agents to a human secretary or personal assistant who is collaborating with the user in the same work environment (Maes, 1994), as that shown in the example at the beginning of the current section. Initially, a personal assistant is not very familiar with the habits and preferences of his or her boss and may not be very helpful. The personal assistant needs to become familiar with the particular working habits of his or her boss and gradually becomes more effective and competent as he or she acquires knowledge. Then, the personal assistant can carry out activities that were initially done by his or her boss, he or she can suggest the boss the execution of tasks, he or she can notify the employer

about interesting situations and make warnings about problems that may arise. In the same way, an interface agent becomes more competent as it interacts with a user and learns about him.

As we will center on adding the capability of performing plan recognition to an interface agent, we will consider plan recognition as the process of inferring the intentions of a *user* based on the tasks he performs in a *software application*. Henceforward, we will use the term “user” to refer the subject being observed, and the term “software application” to refer to the environment in which the subject is interacting.

Although the basic algorithm to accomplish plan recognition seems to be straightforward, when applied to a practical example, we have to consider several issues:

1. Some tasks can be part of many plans (overloaded tasks), for example the user can add a new contact to the address book aiming at sending him an email or aiming at scheduling a meeting with him.
2. There is often more than one way to achieve a goal (multiple plans). A meeting can be scheduled by selecting a slot of time from the calendar view, right clicking, and selecting “Add meeting” or by selecting “Add meeting” from the “Agenda” menu.
3. The user can interrupt the plan he is performing or he can pursue several goals at a time (abandoned and interleaved plans). In the example of the personal assistant presented above, what if before completing the intention the assistant believes the boss has, the boss receives a telephone call and the assistant observes that the boss makes a new entry in his personal diary in the application? Would the assistant consider that the boss belief was wrong? The boss can yet continue editing the event after saving the new entry in the diary.
4. The user can perform tasks that do not belong to their main goal, such as checking the current date (noisy tasks).
5. The uncertainty related to the moment in which the user starts a new plan, and to the goal the user is pursuing itself.
6. The adaptation of the process to the peculiarities of the particular user of the application. This is a key aspect when considering plan recognition in the context of interface agents. Recalling the example of the personal assistant, he or she can learn, for example in which locations the boss schedules working meetings.

In this paper we describe different previous approaches to the plan recognition problem. The work that is most similar to ours is a survey

presented by Sandra Carberry (Carberry, 2001). The aspects she addressed in her survey are plan ambiguity, robustness and efficiency. Plan ambiguity is related to point 1 above meaning that the plan recognizer will have to decide between multiple possible plans that can explain the same set of performed actions. It is also related to the plan library representation that allows a better detection of the user's intention. Robustness in Carberry's survey refers to point 4 above, that is the ability to manage noise in data. Finally, we do not concentrate in the efficiency of existent plan recognition approaches because, as Carberry's also points out, most systems use experimental data provided by simulations and do not test their approaches in realistic scenarios with real users. Our survey can be seen as an update to Carberry's survey, describing and comparing new, mainly probabilistic, approaches. We also put special emphasis on the way each approach represents the user's intentions and on whether it considers user's preferences to adapt the intentions it can detect to a particular user of the software application.

2. What is plan recognition?

The term that has been introduced to describe the process of inferring intentions from actions is *plan recognition* (Schmidt and Goodson, 1978). More specifically, plan recognition can be defined as the task of recognizing the intentions of the user based on the available evidence, that is, users actions, explicit statements about intentions, and users preferences (Kautz and Allen, 1986)

At this point, we shall make clear some terms that we have already introduced, and that we will use along this document:

- *Actions* are events performed by the user in the graphical user interface (GUI). Actions are at the GUI level, for example a click on a button, a text typed in a text field, etc.
- *Tasks* are a piece of work that the user can perform in the application. Tasks are at the application level. An action in the GUI might correspond to a Task in the application level. Examples of tasks in the email domain can be sending an email, entering the subject, adding a new contact to the address book, etc.
- *Intention/goal* is the desire of the user to do something. Intentions are at the user level and are not directly accessible by the agent. The execution of a set of tasks might fit to an intention of the user. For example, arranging a meeting between some of his friends.

- *Plans/recipes* are the set of tasks the user performs to achieve his intention. In the example presented, he should select (or eventually add) from the address book the friends that will take part in the meeting, write a mail to them, select a place and date, etc.

The inputs to a plan recognizer are generally a set of goals the agent expects the user to carry out in the domain, a set of plans describing the way in which the user can reach each goal, and an action observed by the agent. The plan recognition process itself consists in foretelling the users goal, and determining how the observed action contribute to reach it. Plans make up a plan library and can include preconditions, effects and sub goals. The reasoning followed by the process uses this domain dependent information, but the process itself is domain independent.

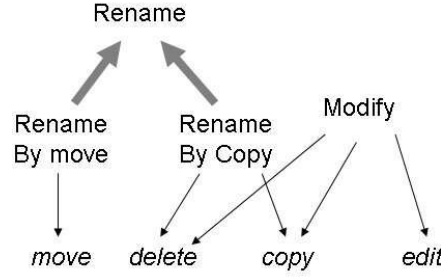
Notice that most of the approaches we will present do not make a distinction between “action” and “task” as we introduced in this section. They consider an “action” as the basic operation of the user. This notion of “action” corresponds to the notion of “task” presented here.

3. Classification of plan recognition

Plan recognition systems can be broadly grouped according to the way they model actions, intentions and plans into *consistency* (or *symbolic*) and *probabilistic* approaches. On the other hand, they can be classified according to the role the user plays in the plan recognition process into *intended*, *keyhole*, and *obstructed* plan recognition. In this section we will detail each of these systems.

3.1. MODELING ACTIONS, INTENTIONS AND PLANS

All plan recognition systems have a *Plan Library* as an input. A plan library is a knowledge base that codifies in some way the agent’s beliefs concerning how the user can reach each particular goal in the domain. Several representations of user’s plans have been used to approach the problem, and several methods applied to infer the intention of the user. These methods can be grouped in two main categories: *Consistency* and *Probabilistic* approaches. Briefly, consistency approaches aim at narrowing the set of candidate intentions by eliminating those plans that cannot be explained by the actions that the user performs. Probabilistic approaches, on the other hand, explicitly represent the uncertainty associated to user’s plans and allow a probabilistic ranking of the user’s intentions. We will present both approaches following an example in an Operating System Domain where the user can follow only two different plans: to rename a file and to modify a file. The user can rename a



$P = \{Rename, Modify, RenameByMove, RenameByCopy\},$
 $Dec = \{ \langle RenameByMove, \{move\}, \{\} \rangle, \langle Modify, \{copy, edit, delete\}, \{copy \prec edit \prec delete\} \rangle \}$
 $Ab = \{ \langle RenameByMove, Rename \rangle, \langle RenameByCopy, Rename \rangle, \{\} \}$

Figure 1. Example of a plan hierarchy

file, in two ways. The first is to rename by copying, which consists in making a copy of the file and deleting the original file. The second is renaming by moving, which is achieved by simply moving the original file. To modify a file, we have to follow three steps: the original file is backed up by making a copy, the original file is then edited and, finally, the backup copy is deleted.

3.1.1. Consistency approaches

The most used representation for consistency approaches are plan hierarchies and consistency graphs. In general, a plan hierarchy is a tuple $H = \langle P, Dec, Ab \rangle$ where P is the set of plans represented in the library. Dec is called the *decomposition hierarchy* and consists in a set of tuples $Dec = \langle p, Ap, Cp \rangle$, where $p \in P$ is the name of the plan, Ap is a set of actions and Cp is a set of constraints concerning these actions such as conditions about temporal relationships among the elements of Ap . Ab is called the *abstraction hierarchy* and consists of a set of pairs $\langle p_1, p_2 \rangle$ where $p_1, p_2 \in P$ with the meaning that p_1 is being abstracted by p_2 and a set of pairs $\langle a_1, a_2 \rangle$ where $a_1, a_2 \in Ap$ with the meaning that a_1 is being abstracted by a_2 . Other plan hierarchy representations omit the abstraction hierarchy.

Figure 1 shows a plan hierarchy corresponding to the operating system example presented previously. Thick arrows correspond to the abstraction hierarchy and thin arrows correspond to the decomposition hierarchy.

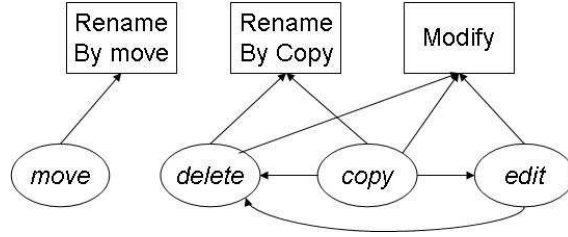


Figure 2. Example of consistency graph

On the other hand, consistency graphs are directed graphs connecting actions, action schemas, and goals schemas. Actions represent observations while the action schemas represent possible unobserved actions. Goal schemas represent goals the user might have. The edges indicate when one action can support another action or goal. Figure 2 shows a consistency graph for the operating system example. Ovals represent action schemas and squares represent goal schemas.

In section 4 we will discuss previous consistency approaches to plan recognition.

3.1.2. Probabilistic approaches

Probabilistic approaches mainly make use of Bayesian Networks and Markov Models. *Bayesian Networks* (or belief networks) (Pearl, 1988) are a popular representation for reasoning under uncertainty because they combine a graphical representation of causal relationships with a sound Bayesian foundation. The directed acyclic graph structure of the network contains representations of both the conditional dependencies and independencies between elements of the problem domain. The knowledge is represented by nodes called random variables and arcs representing the causal relationships between variables. The strengths of the relationships are described using parameters encoded in conditional probability tables. Figure 3 shows the previous example expressed with a Bayesian network.

Markov models have a simple structure (a state transition graph) due to its dependence in the Markov assumption to represent sequences of events. According to this assumption the occurrence of the next event depends only on a fixed number of previous events. Given some previously observed events, the next event is predicted from the probability distribution of events that followed those observed in the past. Figure 4 shows a Markov model of the operating system example.

In section 5 we will discuss previous probabilistic approaches to plan recognition.

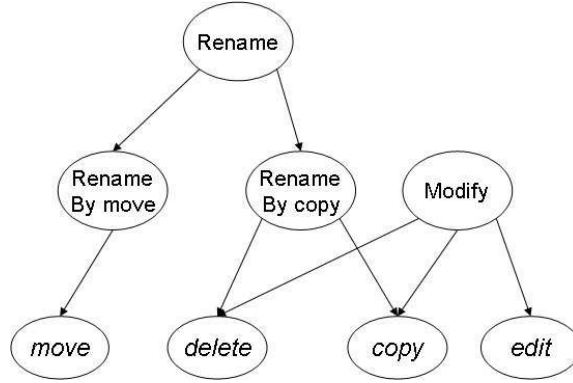


Figure 3. Example of Bayesian network

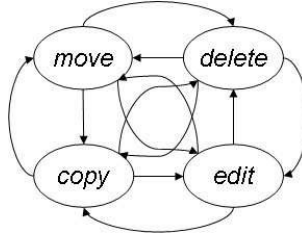


Figure 4. Example of Markov Model

3.2. CLASSIFICATION ACCORDING TO THE ROLE OF THE USER

Other way of categorizing plan recognition has been according to the role that the user plays in the process (Wærn, 1996). According to this classification, we can discern three kinds of plan recognition systems, namely:

- Intended plan recognition, where the user is aware of the process of plan recognition, and actively participates in it. The user can communicate his plans to the system, and the agent can ask the user when facing ambiguous or incomplete information. The user is cooperative and his actions are intended to be understood by the agent.
- Keyhole plan recognition, where the user is indifferent or unaware of the process of plan recognition. Observation, which may be incomplete, is not manifested to the user, as if looking through a keyhole.
- Obstructed plan recognition, where the user is aware and actively obstructs the plan recognition process.

An example of intended plan recognition can be found in a cooperative problem-solving environment. On the other hand in a UNIX console for example, users enter a sequence of commands to achieve a higher level goal and the agent should infer the user's intention by observing the commands he performed. In this case, the agent is following a keyhole plan recognition process. Finally, obstructed plan recognition is applicable, for example, in a chess game, in which participants do not want their opponent to recognize their intentions, and do their best to hide them.

The plan recognition process carried out by an interface agent should be at least keyhole. However, as the user is generally aware of the presence of the agent, and would not obstruct its task, the agent may ask the user in presence of incomplete or ambiguous information. This way, an interface agent can use a hybrid approach to the plan recognition process, but having every moment in mind not to bother the user with many questions. An agent can follow obstructed plan recognition of a user, for example in an application that monitors the user's work. However, we will not consider this kind of agents.

4. Consistency approaches

Consistency approaches face the problem of plan recognition by determining which of an input set of goals is consistent with the observed actions. A goal G is consistent with an observed action sequence A if A might have been executed in service of G .

4.1. KAUTZ'S APPROACH

Kautz (Kautz and Allen, 1986; Kautz, 1987; Kautz, 1991) provides the first formal theory of plan recognition in which plans and goals are represented as an event hierarchy. This hierarchy describes all the behavior that the user might exhibit in some particular domain.

Every observed action is part of one or more top-level plans, and the plan recognition task is to minimize the set of top-level actions sufficient to explain the observed actions. The purposes of these hierarchies are to provide information about the structure of plans themselves, including at least the necessary steps and the possible interdependencies among them, and to represent a partial order in the set of plans that will allow the derivation of more abstract explanations to the observed actions.

Figure 5 shows an example of such an event hierarchy taken from Kautz's thesis.

Each node in the hierarchy represents an event that can occur. Events can be actions, plans or goals without a formal distinction among

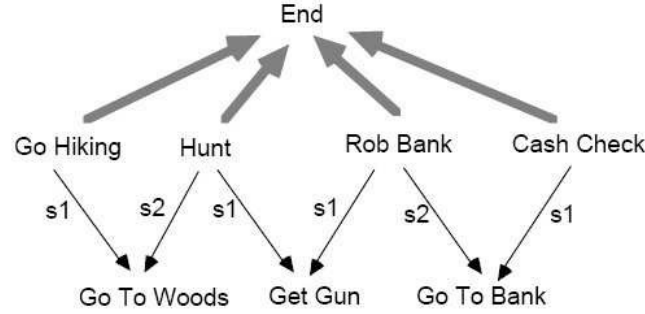


Figure 5. Example of Kautz's Event Hierarchy (Kautz, 1987)

them. Nodes in the hierarchy are connected using two types of connections: *abstraction* (thick gray arrows in Figure 5) and *decomposition* (thin arrows in Figure 5). An abstraction relation indicates that one event is an instance of another (*is a* relation). Decomposition relations describe all the components of a plan event (*has part* relation). Top level goals in the domain are enumerated by adding an abstraction link to a special event called *End* event. The End event represents a goal that the actor is pursuing and that is not in service of any other goal.

Kautz assumed that the observer has complete knowledge of the actor's possible plans and goals and that the actor is performing the minimum number of plans that are consistent with the actor's actions. Kautz also presented a *minimum cardinality* assumption, which states that the fewest possible number of End Events occurs. This assumption allows the recognizer to attribute only as many plans to the actor that are required to explain all the observed actions. For example, if the actor is observed to get a gun and go to the bank, given the event hierarchy presented in Figure 5 and Kautz assumption, the recognizer can conclude that the actor is robbing a bank.

4.1.1. Advantages

The framework proposed is very expressive and allows imposing arbitrary constraints between steps in the plans (not shown in Figure 5), for example that to go hunting one must get a gun before going to the woods. It was the base for many posterior plan recognizers.

4.1.2. Disadvantages

Firstly, Kautz approach ignores priors of plans. Second, his approach is very sensitive to noisy actions. If we observe an action that is not consistent with the current plan, the plan recognizer would abandon the plan it believed the user was pursuing. Lastly, the minimization

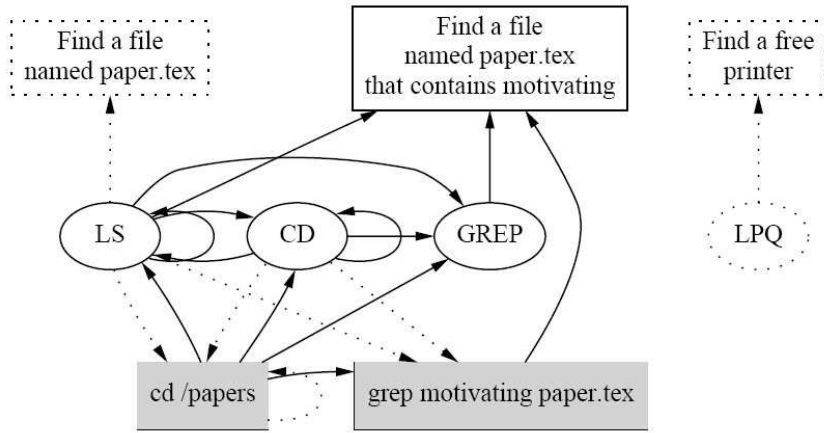


Figure 6. Example of Lesh's consistency graph (Lesh, 1998)

assumption as a principle for abduction does not work in cases where the user can be performing more than one plan at a time.

4.2. LESH AND ETZIONI'S APPROACH

The plan recognizer presented in (Lesh and Etzioni, 1995) uses a graph that represents the relations between the actions and possible goals of the domain. A *consistency graph* is a directed graph in which the nodes are actions, action schemas, and goals schemas. Figure 6 shows a simple consistency graph taken from Lesh's thesis. The consistency graph represents all the plans the actor might be executing. The actions in the graph (shaded boxes) represent the observed part of the actor's plan while the action schemas (symbolized with ovals) represent possible unobserved actions. The goal schemas in the graph (unshaded boxes) represent goals the actor might have. The edges indicate when one action can support another action or goal. The system iteratively applies pruning rules which remove from the graph goals that are not in any consistent plan, given the observed actions. The dotted edges and nodes in Figure 6 represent elements that have been removed.

For example, the Order Rule removes edges between actions (not action schemas): if the recognizer observes the action CD / PAPERS and then LS, the edge from LS to CD / PAPERS can be removed since LS cannot support any previous action.

In a later work (Lesh, 1998) Lesh introduced plan and goal *biases*, which are assumptions about what type of plans and goals people have, to automatically construct a plan library from primitive actions and goal predicates. They split the plan library into two parts: primitives

and biases. The primitives are actions and predicates. The biases are grammars for composing the actions into plans and predicates into goals. They can define, for example, a plan library that contains all shortest plans for goals that can be formed using only two predicates, or a plan library that contains all plans that achieve goals consisting of conjunctions of the input predicates.

Lesh also presented in his thesis an algorithm, called ADAPT, for adapting a goal recognizer to perform well on a sample of behavior. The user's recent behavior is treated as training data, and then a combination of adaptations with which the recognizer performs best on the sample behavior is attempted to be found. ADAPT is an unsupervised learning algorithm in that the sample data is not annotated with the user's actual goals. ADAPT is a recognizer independent algorithm in that it can be applied to any goal recognizer for which there is an available set of adaptations. Two kinds of adaptations are possible in the goal recognizer they propose. The first consists in adding or removing background goals, which are used to filter out actions that are not causally connected to the user's goal. Background goals act as action filters such that any observed action in a background goal is filtered out, or ignored by the recognizer. This way the recognizer is less sensitive to noise. The second adaptation is to add or remove the predicates that the algorithm uses to form goals.

4.2.1. *Advantages*

This approach is similar to Kautz's in the sense that it also keeps track of all consistent explanations, within the plan library, for the observed actions. However, in Kautz's approach, the plan library is an input of the system whereas Lesh and Etzioni construct it automatically from biases. Additionally, they introduce techniques for implicitly representing the library in contrast to Kautz's explicit representation.

They overcome the problem of sensitivity to noisy or spurious actions by introducing a set of "background goals" to filter out irrelevant actions that arise in people's plans or the process of observing these plans. For example, a background goal "know what time it is" could be used to explain an observed execution of the UNIX DATE command and thus this observation would not be used to prune candidate goals.

Other advantages of this approach are that it runs in polynomial time in size of the input, and that the system admits conjunctive goals.

4.2.2. *Disadvantages*

Lesh and Etzioni's approach has the main drawbacks of sacrificing expressiveness in order to automate construction of the plan library. Plan hierarchies enable the specification of arbitrary constraints between

steps of plans, and the designer can arbitrary omit any plan and goals. In contrast, Lesh and Etzioni's require the plan library to contain all and only plans and goals that share an underlying structure, defined by the plan and goal biases. This plan recognition model does not support changes in the plan the user is following nor interleaved plan execution.

4.3. COLLAGEN

COLLAGEN(Rich and Sidner, 1997; Rich and Sidner, 1998; Lesh, 1999; Rich et al., 2001) is an intelligent user interface system that models human-computer interaction based on the concepts of *SharedPlan* (Grosz and Kraus, 1996; Grosz and Kraus, 1999). Grosz and Sidney studied the task of plan recognition as a collaborative process carried by two or more agents. Their theory states that, for a successful collaboration, participants need to have common beliefs about the goals and actions that can be performed and about the capabilities, intentions and commitments of the other participants. COLLAGEN uses an algorithm of plan recognition that exploits the properties of this theory of collaboration:

- The focus of attention: when the user says, for example, “let’s work on email” not only he is proposing to perform an action, but also he is setting a context that restricts the interpretation of new dialogs and actions.
- The use of hierarchical plans, partially developed: the processes of developing, communicating and performing plans are interleaved. As a result, both the input and the output of the plan recognition algorithm are partially elaborated plans. This makes the difference between COLLAGEN and the classical approach of plan recognition that needs to work with complete and correct plans.
- The possibility to ask the user to clarify a doubt, because of the inherent ambiguity, or simply because the required calculus to understand an observed action is beyond the capabilities of the participant.

COLLAGEN uses a set of actions distinguishing between primitive and high level actions. Then, it uses *recipes* to break down non primitive actions in sub goals (like the decomposition relations in Kautz’s approach). Recipes are presented as functions that map an action to a plan to perform that action. Figure 7 shows an example of the inputs taken by COLLAGEN and its outputs.

The inputs to COLLAGEN plan recognizer are the observed action (K in the example), a partial plan $\mathcal{P}$, a focus action $f \in \mathcal{P}$ (in the example,

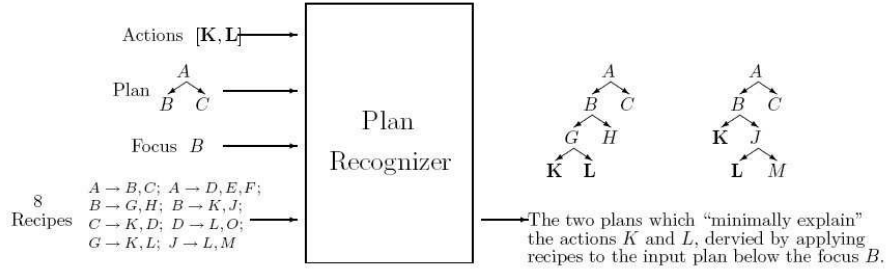


Figure 7. Example of COLLAGEN's inputs and outputs (Rich and Sidner, 1998)

action B), and a recipe library $\mathcal{R}$. The output of the recognizer is a (possibly empty) set of extensions of $\mathcal{P}$ which "minimally explain" the input actions by applying recipes "below" the focus and that includes the user performing actions K and L . If there is only one possible extension, the extended plan becomes part of the new state of the discourse. If there is more than one possible extension, action K is kept and is reinterpreted with the arrival of the next event that could disambiguate the interpretation (which L does not), and so on. The next event could possibly be a clarification question.

Internally, COLLAGEN keeps a focus stack to manage *interruptions*. An interruption is the observation of an action that not contributes to the current partially elaborated plan. An interruption is caused when the user suspends the task he is performing to start a new task. The previous task is resumed when the new task is completed, or it is forsaken. Another circumstances which manifest themselves as interruptions are when the user is pursuing two (or more) goals in parallel, e.g., arbitrarily interleaving steps from both recipes, and when the user performs noisy actions. The focus stack structure can not manage this situation.

4.3.1. Advantages

Recipes can be very expressive, allowing imposing constraints on the temporal order of the steps as well as logical relations among the parameters, such as equality relations. The focus stack provides support for interrupted plans.

4.3.2. Disadvantages

Recipes are constructed manually. The focus stack does not enable the detection of interleaved plans nor noisy actions.

Table I. Comparative table of consistency approaches to plan recognition.

	Kautz	Lesh and Etzioni	COLLAGEN
Type of plan recognition	Keyhole	Keyhole	Keyhole + Intended
Expressiveness	High	Low	High
Supports “noisy” actions?	No	Predefined (background goals)	No
Supports changes in user’s plans?	No	No	Yes, by Pushing and Popping in the Focus Stack
Supports interleaved plans?	No	No	Only “stacked” plans
Considers prior plan probabilities?	No	No	No
Plan Library construction	Manually, Beforehand	Automated from plan and goal biases	Manually, Beforehand

4.4. DISCUSSION

Table I shows a comparison among the approaches studied so far.

4.4.1. *Type of plan recognition*

Both Kautz’s and Lesh and Etzioni’s approaches perform keyhole plan recognition, while COLLAGEN combines both keyhole and intended plan recognition. COLLAGEN takes two inputs that a keyhole plan recognizer does not: a partially elaborated plan and a focus action. These additional inputs simplify the plan recognition task because the recognizer must only extend the input plan, by applying recipes below the focus, just enough to explain the input actions.

4.4.2. *Expressiveness*

As plan libraries in both Kautz’s approach and COLLAGEN are constructed manually, they can be highly expressive. In both approaches it is possible to define arbitrary constraints between steps in the plans and

COLLAGEN allows also the definition of logical relations between the parameters. On the other hand, Lesh and Etzioni sacrifice expressiveness in order to automatically construct the plan library.

4.4.3. *Sensitiveness to noisy actions*

Consistency approaches are characterized by a high sensitivity to spurious (or “noisy”) actions performed while pursuing a goal. Lesh introduced the concept of background goal to filter out those actions. However, background goals are predefined so the agent will be only able to filter a fixed set of tasks considered as “noise”. In COLLAGEN a noisy action can be treated as an interruption. When the user performs an action that is not part of the user’s main goal, it will be put into the *focus stack*. The problem with this approach is that the noisy action will be stacked and the previous plan suspended until the agent realizes that the action will not contribute to any future plan. Then, the action is popped and the previous plan resumed.

4.4.4. *Interrupted and interleaved plans*

The *focus stack* gives COLLAGEN an advantage over Kautz’s approach and Lesh and Etzioni’s approach: the support of changes in the user’s current plan and the ability to resume suspended plans. However, the focus stack does not allow COLLAGEN to detect interleaved plans, that is, it can not detect the situation in which the user is performing actions of more than one plan concurrently.

4.4.5. *Plan library construction*

Kautz assumed that the observer has complete knowledge of the actor’s possible plans and goals and that the actor executes the minimum number of plans that are consistent with the actor’s action. COLLAGEN also considers that the recipe library is complete beforehand. In Lesh and Etzioni’s approach, on the other hand, the plan library is built automatically from assumptions about what type of plans and goals the user will have (what they call biases), at the expense of expressiveness.

4.4.6. *Prior plan probabilities*

In all the approaches presented in this section, the recognizer must consider all possible plans and goals that the observed actions might be part of. The recognizer has to do an exhaustive depth first search in the hypothesis space. If the agent observes an action that is part of many plans, the recognizer must consider all of them and can’t choose among all those possibilities. For example, if the recognizer observes the actor going to the bank, it must consider that he is either going cash a check or going rob it. COLLAGEN performs better in this sense, as it keeps a

partial plan and the focus action and expands the partial plan below the focus action in each step of the plan recognition process.

However, in general, some plans are inherently more likely than others, given that they all correctly explain the observed actions. In the example above, with no further information, cashing a check is a more probable explanation to the action of a person entering a bank. If we see next that person getting a gun, now the robbery is the best explanation given the observed actions. Consistency approaches do not consider the prior probabilities that should to be associated with plans and goals limiting to keeping track of all the possible explanations in the plan library that are consistent with the observed actions.

5. Probabilistic approaches

Another natural way of viewing plan recognition is as a kind of probabilistic reasoning. The plan the recognizer selects as a candidate intention that with the higher probability in light of the evidence at each moment. A major strength of the probabilistic approach is that it allows one to capture the fact that certain plans are a priori more likely than others.

5.1. CHARNIAK AND GOLDMAN'S APPROACH

Charniak and Goldman(Charniak and Goldman, 1989; Charniak and Goldman, 1991; Charniak and Goldman, 1993) built the first probabilistic model of plan recognition. They use a quantifier-free first order language as representation and belief nets for plan inference. The random variables in the belief net are propositions, whereas the root nodes are hypotheses about agent's plan. Charniak and Goldman apply their plan recognition system in the area of story understanding. They apply a set of rules to a sequence of observations using plan knowledge, to automatically generate a Bayesian Network. Then, after incorporating prior probabilities, this network supports the selection of possible explanations of observed actions using the posterior probability of each hypothesis, computed by propagating the values from the evidence in the net. The predicate-calculus like language of this approach is based on collections of actions. Although this representation is adequate for modeling hierarchical action description, as in a story, it does not support other features of plan representation such as sequences of actions.

5.1.1. *Advantages*

The use of Bayesian networks allows the support for noisy actions, interleaved plans and assessment of candidate plans. The networks are automatically built from a quantifier-free first order language representation and probabilities are set by rules.

5.1.2. *Disadvantages*

This approach assumes that we have the plan library represented in a quantifier-free first order language. The network is built from rules over that representation, but it never adapts to a particular user and the network does not learn the probabilities of plans. It is not addressed the way they manage (or not) changes in the user's plan.

5.2. HUBER'S APPROACH

Huber (Huber and Durfee, 1993; Huber et al., 1994; Huber and Simpson, 2004) described methods for directly constructing a Bayesian network from a description of the plans and goals that the actor might have in some domain. They use PRS (*Procedural Reasoning System*) as a general-purpose language for plan specification, which supports sequencing, sub-goals, conditional plans, etc. This allows plan recognition to reuse the same plan representation used for planning. All possible observed actions correspond to variables in the network, as do possible plans and goals. Plan recognition proceeds by fixing the probabilities of the variables corresponding to actual observed actions and then propagating this information through the graph. In this way, one can compute the posterior probabilities of each possible goal the actor might have.

5.2.1. *Advantages*

As before, support for noisy actions, interleaved plans and assessment of candidate plans is given by the representation. Bayesian networks are built automatically from a PRS representation of user's plans.

5.2.2. *Disadvantages*

A previous PRS representation is assumed. Probabilities must be hand coded. It is not addressed the way they manage (or not) changes in the user's plan.

5.3. THE LUMIÈRE PROJECT

The Lumière/Excel project (Horvitz et al., 1998) is centered in the use of probabilities and utility functions to provide assistance to Microsoft

Office 97TM users. The core of this project is in the Bayesian user model that captures the uncertain relations among the goals and the needs of a user and the observations about the state of the program, sequences of actions through time and words in the user's queries. *Goals* are target tasks or sub-tasks at the focus of the user's attention. *Needs* are information or automated actions that will reduce the time or effort required to achieve the goals.

Horvitz states that Bayesian models can be effective in the diagnosis of user's needs, and that can provide useful improvements to legacy systems embedded with these kinds of programs. This research is centered in understanding the needs and behaviors of the users when they face a problem with the software application, and in the utility of offering assistance. Interface events are transformed in observations with a time stamp. These observations are the input to the Bayesian model and the probabilities distribution is derived over the needs of the user. The Bayesian information retrieval system recognizes approximately 600 terms and considers the probabilistic relevance of these terms to the areas of assistance considered by Lumière/Excel. If a query was made, posterior probabilities from the events in the system are combined through a weighted multiplication to yield a final posterior distribution over needs. Figure 8 shows a partial structure of the Bayesian user model used for inferring a user's needs in the Excel spreadsheet application. Beyond reasoning about the probability distribution over user's problems, the system also infers the likelihood that a user needs assistance at the present moment. This probability is used to control the autonomous display of assistance when the system is running in the background.

In addition to analyzing real-time events, Lumière/Excel maintains a persistent competency user profile that stores sets of key tasks completed and help topics reviewed. This profile can be used to update the probability distribution over needs. In a more general approach, a version of Lumière was created that enables experts to author special competency variables in the Bayesian user model. These variables have values that reflect the nature and number of times a user demonstrates successful manipulation of different functionalities. These variables can be integrated into the model as any other variable but their states are stored in a persistent user profile.

5.3.1. *Advantages*

This approach also considers the utility of offering assistance to the user embedded with the plan recognition system. The probabilities of the network are updated by keeping a persistent profile of the tasks completed by the user and the help topics reviewed.

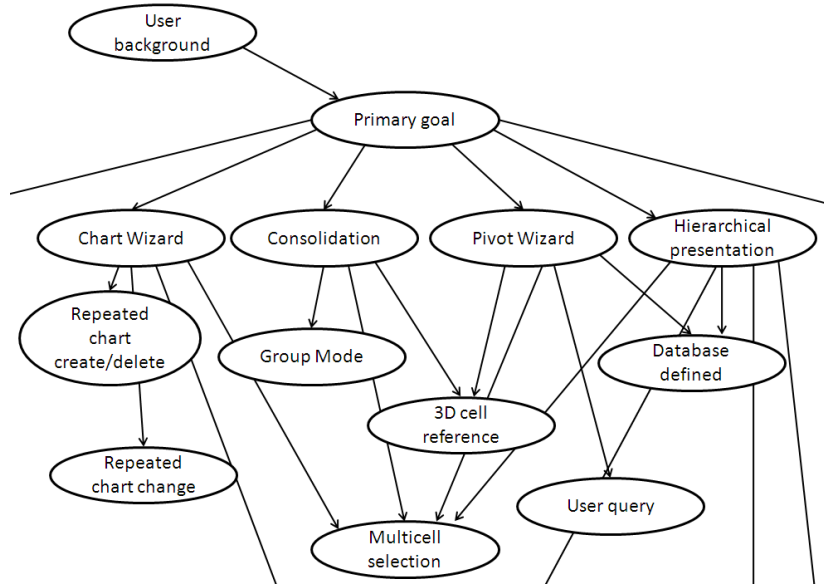


Figure 8. Partial structure of Horvitz's Bayesian user model for inferring the needs of the user of Excel spreadsheet (Horvitz et al., 1998)

5.3.2. Disadvantages

The network is manually built. It is not clear how they manage changes in the user's plan.

5.4. CORE INTERFACE AGENT ARCHITECTURE

Brown (Brown, 1998) also uses Bayesian networks to build a user profile that allows the detection of his intentions considering the utility of offering assistance. This approach (the *Core Interface Agent Architecture*, CIA) is based on the belief that what the user intends to do in an environment is the result of the stimulus (i.e. the events) that occurs in the environment, and of the goals he is trying to reach as a reaction to those stimuli. Goals can be explicit (arranging a meeting, for example) or implicit (reducing work overload). To reach a goal, a user has to perform certain actions. Goals are made of a set of actions, with pre and post conditions. Preconditions include both events that are directly observable in the environment (like the user clicking in a button labeled "Add event") and events that are indirectly observable (such as increase cognitive load of the user). These preconditions cause the user to follow a goal.

Brown uses a directed acyclic AND/OR graph that shows the causality among preconditions, goals and actions (Figure 9). The network has

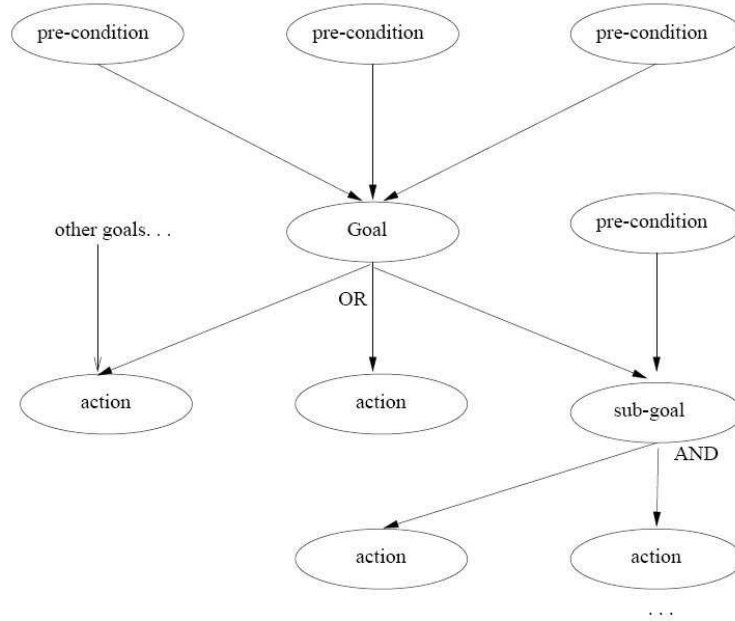


Figure 9. AND/OR graph representation of Brown's user model (Brown, 1998)

three different kinds of random variables: goals, actions and preconditions. Preconditions are in the root of the tree and actions in its leaves. Each action node has an associated utility function to determine the expected utility of offering assistance to reach the parent goal. Precondition nodes are used to catch the events of the environment.

Brown's architecture, uses a multi-agent approach to fit the user model to a particular user. He defines four requirements (adaptability, autonomy, collaboration, and robustness) that interface agents should have. The ability of interface agents to fulfill those requirements is measured by means of metrics for each one and a utility function. The use of that utility function has the purpose of determining when the user model is not accurate, and how to revise the model. This multi-agent approach with correction adaptation agents allows the interface agent to determine dynamically the best correction to the user model. Each correction adaptation agent has a possibly unique method to correct the user model of the interface agent. These agents engaged in a bidding process to recommend changes to the ailing interface agent user model, who serves as a manager agent. The interface agent is responsible for determining when a contract is available, announcing the contract to be filled, receiving bids from the bidder agents, and finally accepting or rejecting the bids based on their utility.

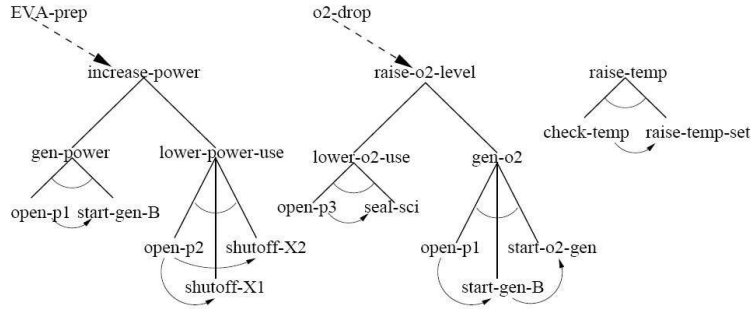


Figure 10. Geib and Miller's hierarchical plan library representation (Goldman et al., 1999)

5.4.1. Advantages

Among the advantages of this representation are that goals abstraction allow the design and detection of higher level goals in the search of lower level goals, evidence can be added and removed easily and intuitively (in the form of pre and post conditions) as the user interacts with the system, pre and post conditions for goals and actions can be explicitly indicated, and keyhole plan recognition is made easier by enumerating explicitly the actions that make up goals.

5.4.2. Disadvantages

Brown uses a multi-agent approach with correction adaptation agents engaged in the task of keeping an accurate user profile. Although this system seems promising, he does not address the way the correction adaptation agents are implemented.

5.5. GOLDMAN, GEIB AND MILLER'S APPROACH

Goldman, Geib and Miller (Goldman et al., 1999; Geib, 2004; Geib and Goldman, 2005) presented a new abductive, probabilistic theory of plan recognition based on plan execution. They use a simple hierarchical (task decomposition) representation of plans that provides recipes for achieving goals, as that shown in Figure 10. Horizontal arrows represent temporal restrictions. Tied arcs from a parent node to a child node represent "and" nodes, while untied child represent "or" nodes, or alternative plans to achieve the (sub) goal represented by a node.

In their model of plan execution, initially the user has a set of goals and chooses a set of plans to execute to achieve those goals. The set of plans chosen determines the set of *pending* primitive actions. Then the user will repeatedly execute one of the pending actions and generate a new set of pending actions from which further action will be chosen.

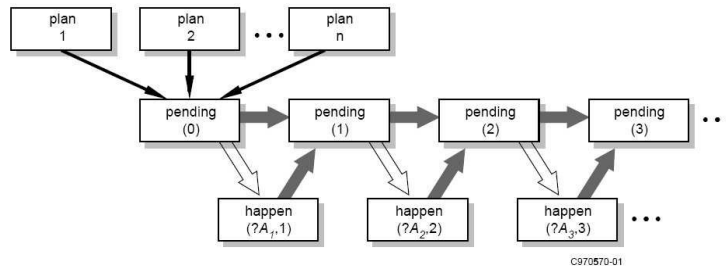


Figure 11. Goldman, Geib and Miller's model of pending set generation (Goldman et al., 1999)

The new pending set is generated from the previous set by removing the last executed action and adding the actions that become enabled by it execution. Actions become enabled when their required predecessors are completed. This process is illustrated in Figure 11.

They define an explanation of a set of observations as a minimal forest of instances of plan trees with expansions chosen for “or” nodes sufficient to allow an assignment of each observation to a specific basic action in the plan. The algorithm first computes the complete and covering set of possible explanations; second, it computes the probability of each of the explanations; finally, it computes the conditional probability of the given goal on the basis of the probability of the explanations.

5.5.1. Advantages

This approach handles partially-ordered plans, interleaved plans, the effect of context in plan choice and is able to treat the execution of actions for its own sake. It also supports missing observations, and noisy actions that do not contribute to a goal.

5.5.2. Disadvantages

The number of explanations can grow exponentially in the number of root goals that share a common unordered prefix of actions. These “shared leaders” appear in domains where many different plans for different goals have a *common prefix* of actions, and have a significant impact on the algorithm's run-time. Plan libraries without ordering constraints represent the worst case for the algorithm performance.

5.6. PYNADATH AND WELLMAN APPROACH

Pynadath and Wellman (Pynadath and Wellman, 1998) proposed a plan recognition method that is both probabilistic and based on parsing. They represent plan libraries as probabilistic context-free grammars

(PCFGs) and extract Bayesian networks from the PCFGs to interpret observation sequences. A PCFG is a tuple $\langle \Sigma, N, S, P \rangle$, where Σ and N are disjoint sets specifying the terminal and non-terminal symbols, respectively, $S \in N$ being the start symbol. P is the set of productions, which take the form $E \rightarrow \xi(p)$, $E \in N$, $\xi \in (\Sigma \cup N)^+$, and $p = Pr(E \rightarrow \xi)$, the probability that E will be expanded into the string ξ . The probability of applying a particular production $E \rightarrow \xi$ to an intermediate string is conditionally independent of what productions generated this string, or what production will be applied to the other symbols in the string, given the presence of E . Therefore, the probability of a given derivation is simply the product of the probabilities of the individual productions involved. They define a *parse tree* representation of each such derivation as for non-probabilistic context-free grammars. The probability of a string in the language is the sum taken over all its possible derivations.

In later work (Pynadath and Wellman, 2000), Probabilistic State-Dependent Grammars (PSDG) are presented. PSDG can be described as the PCFG, augmented with a state space, and a state transition probability table for each terminal symbol of the PCFG. In addition, the probability of each production rule is made state dependent. As a result, the terminal symbol now acts like primitive actions and the non-terminal symbol chooses its expansion depending on the current state.

5.6.1. *Advantages*

Grammatical representations are generative and modular, providing an appealing class of languages for specifying pattern-generation processes. Bayesian networks representing the distribution over all parses of strings are automatically generated from a probabilistic grammar. The PSDG language contributes a new representation that exploits the partition between plans and state that exists in most plan recognition domains. PSDG are a very expressive representation for plan libraries.

5.6.2. *Disadvantages*

The major problem of parsing as a model of plan recognition is that it does not treat partially ordered plans or interleaved plans. Punctually, partial ordering would cause an explosion in the size of the grammars. PSDGs arise as a possible solution to this problem but it is difficult to define a probability distribution for a PSDG. Furthermore, the algorithms cannot exploit direct evidence about plans. On the other hand, PCFGs needs the complete set of observations to work with and, in plan recognition, observations arise during the execution of the plan, before the entire sequence is available. Finally, the time and space complexity

is quadratic in the number of instantiations of the unobserved state variables. This cost is potentially prohibitive, since the number of such state instantiations grows exponentially with the number of unobserved state variables.

5.7. INCREMENTAL PROBABILISTIC ACTION MODELING

Another probabilistic technique used to ascribe users intentions are *Markov Models*. Davison and Hirsh (Davison and Hirsh, 1998) approach the task of predicting the next element in a sequence formed by nominal elements (both non ordered and no numeric). Considering the simplest Markov assumption that each command depends only on the previous command (that is, patterns with a length of two commands, in that way that the previous command is the state), they use statistical data to count the number of times that each command was followed by each other command, and he estimates the probability of a future command.

The algorithm is called Incremental Probabilistic Action Modeling (IPAM) and is implemented using a simple structure of an N by N matrix that shows the probability of passing from one command to the next. At first the matrix is empty and is updated after each command. In an empty table all command are equally likely. When the first command c_i is observed, a new row is added for that command, with a uniform distribution of probabilities. With the next command c_{i+1} , a new row is also added with uniform distribution, but the first row is updated, as we have seen command c_i followed by command c_{i+1} , by multiplying every elements of the row by a constant $0 \leq \alpha \leq 1$, and the likelihood of observing c_{i+1} is incremented by adding $(1 - \alpha)$. This way more recent commands are emphasized at the expense of older actions, but the sum of the probabilities of each row is always 1. A value of 0 of the alpha constant means that the algorithm will always predict the more recently observed, while a value of 1 is related to an algorithm that never changes its probabilities.

5.7.1. *Advantages*

This algorithm is easy to implement and very quick. It can learn patterns in the user's behavior with any information neither of him nor of the application.

5.7.2. *Disadvantages*

It can only predict the next action of the user, given the previous one. There is no way of predicting the user's plan.

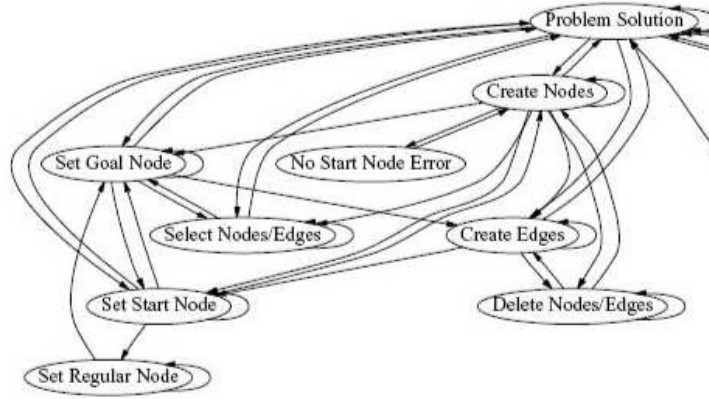


Figure 12. Example of Markov model learned by Gorniak's approach (Gorniak, 2000)

5.8. OFESI AND ONISI

On the other hand, Gorniak (Gorniak, 2000) builds automatically a Markov model based on the history of observed actions from the user. He presents a state identification method tailored for building humanly understandable models of the use of the application. Figure 12 shows the model he obtained for a particular application; the figure represents the model exactly as recorded, except that meaningful names were given to the states. Then, he applies an algorithm called OFESI (Off-Line Explicit State Identification) to refine the model by checking whether they fulfill the Markov assumption. If a state does not fulfill this assumption, OFESI replaces it with new sub-states that are more Markovian by grouping history sequences preceding the state according to how much the resulting sub-states reveal about actions occurring in the parent state. Applying OFESI results in a detailed model of the use of the application. To predict the next action of the user, he applies the ONISI (On-Line Implicit State Identification) algorithm that implicitly refines the base model states by weighing possible actions by the lengths of matches to the history preceding the current state. While Davison and Hirsh only consider the previous action to predict the next, Gorniak leaves the length of the previous action sequence as an input to the algorithm.

5.8.1. Advantages

ONISI automatically vary the length of patterns it identifies. It can build a model of the way a user utilizes an application from a history of his previous actions. Gorniak's is a promising approach to build a base

model of user's behavior that can serve as a base to help designers to build other informed models.

5.8.2. *Disadvantages*

It is necessary to keep memory of a quite long history of user's actions, and the complexity of the search largely depends on the length of the action sequence considered.

Despite both Markovian approaches can work from the scratch (without any previous structure or knowledge of the application), they only assign a possible next action given the previous history of the user's behavior. However, they do not consider that the user can perform a set of actions to achieve a high level goal.

5.9. BUI'S APPROACH

A more recent approach that uses Markov Models is that proposed by Bui (Bui et al., 2002). He proposed a framework for online probabilistic plan recognition based on the Abstract Hidden Markov Models (AHMM). A AHMM is an stochastic model for representing the execution of a hierarchy of contingent plans. Plans in this model are termed *policies*. Then, an extension allowing policies to have "memory" (AHMEM) was introduced (Bui, 2003) to enable a richer set of hierarchical probabilistic plans to be represented.

In Bui's approach the plan of action of an agent is modeled as a policy that prescribes how the agent would choose an action at each state of the model using a transition function. A policy can be seen this way as a Markov chain through the state space. N levels of policies are allowed to model the user behavior. Policies at each level are called Abstract Policies and at level 0 we have the actions the user can perform as abstract policies themselves.

The dynamic process of executing an abstract policy is called an Abstract Markov Model and is suitable for representing behaviors that are guided by goals with different levels of abstraction. Each policy in a AMM can be seen as a plan being performed to reach a particular goal. A policy specifies the course of actions at all applicable states being more similar to a contingent plan than to a plan itself. When a policy ends, there is no guaranty that the desired goal has been achieved.

5.9.1. *Advantages*

A very rich class of plans can be modeled using this approach due to the ability of structuring any number of levels of policies. The AHMEM increases further the expressiveness of the model including those plans that can be modeled by Pynadath and Wellman PSDGs. Uncertainty

is considered both in the planning and plan observation process. Noisy tasks can be managed by making the state “hidden”, similar to the hidden state of a Hidden Markov Model (Rabiner, 1990).

5.9.2. *Disadvantages*

Although it would be feasible to use techniques for learning the parameters of an AHMM from a database of observations, it is not implemented. It is assumed that the user is following only one top-level policy, not considering the case that the user can perform several policies concurrently.

5.10. DISCUSSION

Table II resumes and compares the approaches analyzed in this section.

5.10.1. *Type of plan recognition*

All the approaches analyzed in this section tackle the problem of keyhole plan recognition.

5.10.2. *Expressiveness*

The first two approaches that we have analyzed using Markov models are lowly expressive. While Horvitz, as an example, can guess from the user’s actions that he is likely trying to define a custom chart in Excel, Gorniak can only say that the user and the application are currently in a state which is represented through a set of action sequences leading to it and a probability distribution over actions and transitions occurring in it. Moreover, Gorniak approach is unable to model the application and the user to the level of detail needed for a sophisticated plan recognizer. Another disadvantage of both Davison and Gorniak approaches is that they have the limitation of not considering high level plans, being restricted to the prediction of the next task the user will perform. This problem is solved by Bui’s AHMM which allows representing the execution of a hierarchy of contingent plans, which are called policies.

Pynadath and Wellman PSGDs are more expressive than the AHMM as the last is not able to represent an uninterrupted sequence of plans and actions. AHMM solve this problem being able to “remember” which subplan in the sequence is being performed at any time including the plan libraries that PSGD is able to represent.

Approaches based on Bayesian networks allow an easy representation of causal influence between tasks and plans. A point that makes Bayesian networks less expressive than Bui or Pynadath and Wellman’s approaches is the fact that there we cannot represent an order in the execution of tasks. In other words, if we enter as evidence in a Bayesian

Table II. Comparative table of probabilistic approaches to plan recognition

	Charniak & Goldman	Huber	Horvitz	Brown	Goldman, Geib & Miller's	Pynadath & Wellman	Davison	Gorniak	Bui
Type of plan recognition	Keyhole	Keyhole	Keyhole	Keyhole	Keyhole	Keyhole	Keyhole	Keyhole	Keyhole
Expressiveness	High	High	High	High	High	High	Low	Low	High
Supports "noisy" actions?	Yes	Yes	Yes	Yes	Yes	Predefined	No	No	Yes
Supports changes in user's plans?	Not men- tioned	Not men- tioned	Not men- tioned	Yes (fad- ing func- tions)	Yes	Not men- tioned	-	-	Not men- tioned
Supports interleaved plans?	Yes	Yes	Yes	Yes	Yes	No	-	-	No
Considers prior plan probabili- ties?	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes
Plan Library construction	From rules.	Structure from PRS descrip- tions. Probabili- ties hand coded	Manually, before- hand	Manually, before- hand	Manually, before- hand	Bayesian networks automatic from proba- bilistic gram- mars. Gram- mars, manually.	From Scratch	From Scratch	Manually, before- hand.

network the same set of tasks but in two different orders, the result of the belief update will be the same. We believe that this is not a main limitation, as causal arcs can be added to “force” the representation of a sequence. For that reason we marked Bayesian network approaches expressiveness as “high”. However, adding that kind of arcs increases notably the number of parameters to be estimated in the model.

5.10.3. *Sensitiveness to noisy actions*

The support for noisy actions came along with Bayesian approaches, as the observation of such kind of action does not discard previous candidate plans. If the user performs a noisy action, no evidence will be entered in the network if the task is not part of any modeled goal. If the noisy action is part of some goal, there will be an increase in the probability that the user is performing that goal, but no further evidence will support that fact. A problem here is when the evidence in a noisy task is retracted and it depends on in which moment the evidence in any node is retracted. This issue is directly related to how each approach manage changes in the user’s current plan (section 5.10.4).

In contrast, as Markov models work with state transitions, an eventual noisy action would cause the transition to a possible incorrect state. This problem can be solved in Bui’s approach by making the state hidden. On the other hand, although in Pynadath and Wellman approach it is easy to create rules that filter out noisy actions, what is considered a noisy action must be predefined when defining the grammar (in a similar way as *background goals* of Lesh and Etzioni approach presented in section 4.2).

5.10.4. *Interrupted and interleaved plans*

For the same reason above, Bayesian models allow the execution of interleaved plans: evidence of different plans can be concurrently entered in the network. However, the only approach which explicitly states how to deal with changes in the user’s goal is Brown’s CIA architecture. His architecture supports fading functions for each observation, which can be time-based (evidence is relevant for a specified time) or queue-based (only the N newest observations are relevant). Both Davison and Gorniak approaches can only predict the next command given the previous performed actions. For this reason, we can not analyze the possibility of changes in user’s plans or the support for interleaved plans.

It is not mentioned in Pynadath and Wellman and Bui’s approaches how they manage transitions in the user’s top level goals and none of this approaches gives a direct support to recognize interleaved plans.

5.10.5. *Plan library construction*

Both Gorniak and Davison's approaches differ from the other techniques in the sense that no previous knowledge of the application is given to the plan recognizer. In these approaches the model is simpler than in Bayesian approaches but it is built automatically as the user utilizes the application. The models of the other presented approaches need to be manually designed or automatically obtained from previous knowledge of user's possible plans and actions (such as PRS descriptions). This fact gives the simpler Markov models the advantage of working without any previous information of the user or the application, but the obtained models are not human understandable.

Although it is not addressed in the approaches we analysed, learning techniques can be used to learn the parameters of Bayesian approaches and Bui's Markovian approach. There also exist many algorithms for learning the structure of a Bayesian network from data (Cooper and Herskovits, 1992; Buntine, 1991; Peter Spirtes and Scheines, 1993) but as the Bayesian models used for plan recognition have a special structure (which is different for each approach), classic learning algorithms cannot be used directly.

5.10.6. *Prior plan probabilities*

As this section was devoted to probabilistic plan recognition, all the presented approaches consider probabilities in their core.

6. User's preferences in plan recognition.

When a plan recognizer is to be adapted to a particular user of a software application, a proper model of his preferences is crucial because preferences are the reasons why a user acts in a particular way when facing a certain situation (Bauer, 1996). For example, a user of a scheduling application can prefer organizing events in his house when the participants are his friends or when it takes place in the afternoon, and in the club when it takes place in the morning. Not knowing about the user's preferences hinders the plan recognition process, especially if the behavior of the user deviates from what is typically to be expected in a domain. So, an agent provided with plan recognition and user modeling capabilities can detect patterns of behavior that are particular to each different user of the application.

Machine learning techniques are a common and the more widely used method to build user models implicitly. The main reason of the popularity of these techniques is that user models can be obtained

incrementally, usually without intervention of the user, by observing the user while he performs his tasks.

In this section we will overview some previous approaches that take user preferences into consideration when ascribing the user's intention.

6.1. MAES'S APPROACH

The most popular heuristic for knowledge acquisition is statistical correlation. That is to say that the actions that the user performed in a particular situation are good candidates to test again under the same situation. For example, Maes stated that situation-action preferences provide relevant information for guiding the plan recognition process. She used case-based reasoning (CBR) (Stanfill and Waltz, 1986) to learn situation-action pairs in applications in the domains of electronic mail, appointment scheduler and web news. Case-based reasoning is a machine learning technique based on frequencies. Situations are represented with vectors of relevant features in the domain of the system. For example, in the electronic mail domain, this vector can include the sender and addressee of the message, the words in the subject field, the length of the message, priority, etc. The relevant features are determined in the design phase. While the user interacts with the applications, situations (determined by its features) are stored along with the action taken by the user. The interface agent uses these raw data of situation-action pairs to offer assistance to the user based in the previous most similar situation. The frequency of the pairs determines its applicability. Statistical correlation is made off-line. Maes uses two thresholds to detect when the statistical correlation is strong enough to allow the agent to act on the users behalf.

For example, if an email agent observes that the user reads a new mail which recipient is the mailing list "intelligent-interfaces", it can predict that his intention is "Move the mail to the 'int-int' folder. This kind of intention could not be detected without having previous knowledge about the user.

6.1.1. *Advantages*

Few past situations are often enough to provide assistance to the user, as a new situation will be classified according to the most similar previous situation.

6.1.2. *Disadvantages*

The use of CBR keeps only raw data in a data base in the form of situation-action pairs. This way, a user has to perform a set of actions before the plan recognizer could take any decision. Moreover, situation-action pairs are restricted to one step predictions, preventing it direct

application to the plan recognition process in which evidence is gathered in different steps of observation. To compare an action with a situation, it has to be completely specified. In case-based reasoning each feature in the vector is relevant by default. On the other hand, as the agent gains more experience by storing more situations, time of finding the more similar situation increases, as well as the space of storage required.

6.2. BAUER'S APPROACH

Bauer (Bauer, 1995), following the idea of Carberry (Carberry, 1990), presented a formalism for the task of assessing the various hypotheses of a plan recognizer by taking into account the user's individual preferences. He used an alternative but related approach that reason about uncertainty using *Dempster-Shafer theory* (Shafer, 1976). Dempster-Shafer theory can make a distinction between the lack of evidence for a proposition from knowing evidence against the proposition. He analyzed the numerical hypothesis assessment of *sets* of hypotheses by interval-valued assessment functions instead of the classical point-valued probability functions (those present in Bayesian networks). The same as in Bayesian approaches, Bauer also used an explicit set of input uncertainties to help select among consistent explanations. However, the hypotheses that are consistent with the actions of the user are provided by an *external* plan recognizer, and they are then assessed by taking into account the user's preferences. Moreover, Bauer (Bauer, 1996) researched the acquisition of user preferences for plan recognition domains in which users typically have pattern of actions when they follow a particular goal. He viewed these repeated patterns as preferences, and used Dempster-Shafer theory to represent preferences deduced from previous activity. He uses ID3 decision trees (Quinlan, 1986), an off-line inductive machine learning technique, to learn classes of situations based on user actions. Bauer's approach runs the plan recognizer on a set of input examples from a typical user, and then gathers statistical data based on the results of running the plan recognizer on the entire observable behavior in each episode. For example, Bauer's techniques can learn that a particular computer user will save his email with 80% certainty, unless the email is from his manager, in which case he deletes it 90% of the times. This analysis enables a plan recognizer to reach the same conclusions with fewer observed actions.

6.2.1. Advantages

Bauer's approach adapts the probabilities of each plan to a particular user of the system. Furthermore, it can detect patterns in the behavior

of a user using ID3 decision trees. Decision trees are easy to interpret by a user as a way of explaining the behavior of the agent.

6.2.2. *Disadvantages*

The main problem of using decision trees to “classify” the intention of the user according to some attribute of his actions, is that they can only ascribe one possible intention given those attributes, completely discarding other options.

6.3. MARAGOUDAKIS ET.AL. APPROACH

Another work on acquiring domain knowledge by probabilistic reasoning is presented in (Maragoudakis et al., 2003b)(Maragoudakis et al., 2003a). This work is based on inferring intentions for actions from questions in an educational dialog engine.

The majority of existing dialog systems that provide informational services expects standardized, pre-defined queries from the user. By following this approach, the quality of the interaction is deteriorated and circumscribed in narrow semantic limits, lacking of any mixed-initiative notion. In such systems, domain knowledge is typically handcrafted by an expert during the design phase and should be as robust as possible to the potential user’s question variations. Such handcrafting of knowledge bases is infeasible for grappling with the linguistic (both lexical and semantic) diversity of a domain.

As plan recognition ability is strongly related to the richness of the domain knowledge representation, Maragoudakis et.al. approach aims at the automatic generation of such knowledge from data. They utilize automatic encoding of the semantic model of the application, based on learning Bayesian networks from past user questions. Both the structure of the networks and the conditional probability distributions are computed automatically from dialog corpora. This framework is Bayesian in that they start from a causal theory of how the agent’s mental state causes its plan, executing its plan causes activity, and they reason from observed effects to underlying causes. A probabilistic model for modeling semantic disambiguation is defined over a search space $H * T$, where H denotes the set of possible lexical context that could be identified within an input query $\{h_1, \dots, h_k\}$, or input variables and T denotes the set of the allowable semantic interpretations of that question $\{t_1, \dots, t_n\}$. To manage the observation of new words that were not seen in the training dataset, their semantic role is estimated by similarities with known words. This is done by looking into a database of lexicon-semantic similarities, acquired via the analysis of the contextual behavior of words in larger collections of texts on the domain of interest.

For example, in the question “What could the active substance lincomycin cause to a pregnant woman?”, they distinguish five lexical elements which are “active substance”, “lincomycin”, “cause”, “pregnant” and “woman”. These elements characterize the intention of the user which is to receive information on the side effects of the active substance “lincomycin” during the gestation period.

6.3.1. *Advantages*

Network structure and probabilities are automatically inferred from data. Knowledge about similarities between words allows the system to take into account words that were not observed during the training phase. The use of Bayesian networks to model human sentence understanding enables the quantitative evaluation of the impact of different independence assumptions in a framework of uniform structure, and develops an on-line mechanism for interpretation, where partial input corresponds with partial evidence on the Bayesian Network. Furthermore, the impact of highly structured linguistic knowledge is straightforwardly modeled with conditional probability tables.

6.3.2. *Disadvantages*

The training dataset must be manually annotated.

6.4. CHEN ET.AL. APPROACH

Chen (Chen et al., 2002) presented a method to model and inferring user’s intention using data mining. They make the distinction between action intention (low level, such as mouse click, keyboard typing and other basic actions performed on a computer) and semantic intention (what the user wants to achieve at high level, which may involve several basic actions on a computer to accomplish it). They work in the domain of a web browser, and mainly focus on predicting action intention based on the features they extracted from the user interaction such as user’s typed sentences and viewed content.

First, they extract two linguistic features (keyword and concept features) for intention modeling. Linguistic features are features in the text that may indicate a user’s intention. Then association rules are used to mine proper concepts of corresponding keywords. These rules represent the association relationship between features and intentions, e.g. given a feature, the association indicates whether the user intends a click action. Furthermore, they use Information Gain (Yang and Pedersen, 1997) to select most discriminative features (both keywords and concept) to reduce the size of dictionary and improve the performance when the training set is small. Finally, a modified Naïve Bayes classifier is used

to model the user's action intention on a computer. The model can be trained incrementally and used to predict the user's next action.

For example, in a Web browser domain, they extract a set of linguistic features from the text typed or used by the user. Then, the prediction module calculates the probabilities of all predefined user intentions and chooses the one with the maximum probability. In this domain, they can predict five user's actions: *browse* (view a Web page), *click* (follow a hyperlink in a Web page), *query* (type query words in a search box), *save* (save pages or the objects in a page) and *close* (close the Web browser window).

6.4.1. *Advantages*

The use of Information Gain to select the most useful features can dramatically reduce the feature set. The modified Naïve Bayes classifier allows the incremental learning of the user model.

6.4.2. *Disadvantages*

They only focus on the prediction of action intentions, that is, the prediction of the next action of the user based on the document he is browsing or what he had typed in a web browser. Plan structure is not considered by this approach.

6.5. DISCUSSION

In regard to acquisition of user's preferences, memory based techniques (as CBR, used by Maes) keep only raw data in a data base in the form of situation-action pairs, while inductive techniques (such as decision trees used by Bauer, or Bayesian networks) try to create a higher level abstraction from situations. Moreover, situation-action pairs are restricted to one step predictions, preventing its direct application to the plan recognition process in which evidence is gathered in different steps of observation.

There are further deficiencies in the situation-action pairs approach. Firstly, determining the relevant features of the domain that allow the situations modeling is a critic task. In the restricted domain of news reading, it is a tractable problem. However, in many domains this restriction makes situation-action pairs an inadequate technique for the problem in question. In addition, to compare an action with a situation, it has to be completely specified. In case-based reasoning each feature in the vector is relevant by default. Designers of user models have to be capable of specifying a situation without ambiguity.

Secondly, a user has to perform a set of actions before the plan recognizer could take any decision. This problem can be solved using

additional acquisition techniques like stereotypes of users that provide a priori knowledge about the user. In this way, Ardisonno and Sestero (Adisorno and Sestero, 1996) used stereotypes and heuristic analysis of the user's previous actions to obtain beliefs about the user that would help to identify implausible plans.

Furthermore, situation-action pairs ignore the correlation between actions (that is to say, behaviors) and the goals pursued by the user. A single action to be performed is assigned to a situation. However, in many situations, the user can perform a set of actions to achieve a high level goal. Simple situation-action pairs can not model situation-goal-actions like cases.

Finally, although most user models that use situation-action pairs allow machine learning techniques to extend the behaviorism model of users by adding new pairs and/or updating the statistical frequencies, they fail in capturing the inherent uncertainty of predicting user intentions.

7. Final comments

Both strict consistency and probabilistic approaches we have analyzed are domain independent and can lead to accurate predictions providing that the plan library is complete in the first case, and that the probabilities are correct in the second. Both have also the disadvantage of considering all the possible plans in the library given the observations. However, if observations so far can not distinguish between a set of possible intentions, probabilistic approaches can find the most probable one, while consistency approaches can not select between them and have to wait for a single consistent explanation.

A drawback of using Bayesian inference is that the result of the probabilities update is not sensitive to the order in which the actions are observed. In other words, if the actions would have happened in a completely different order, the probabilities estimation shown by the Bayesian network would have been exactly the same. This is an important drawback in some applications in which the order in which the tasks are performed is important. On the other hand, strict consistency approaches are very sensitive to noisy actions. Though, if an unexpected action occurs in the middle of the execution of the plan, the plan recognizer could wrongly exclude the plan as consistent with the user's behavior.

Another problem with Bayesian networks is that, as plan recognition is an incremental process, when we add evidences as we observe actions

of the user, the belief update process takes an exponential time in the number of entries of the CPT.

On the other hand, simple Markov models are limited in the sense that they can only predict the next action the user will perform. They do not consider any kind of plan to achieve the user's goal. Nevertheless, they have the advantage of having a way to build them automatically without any knowledge about the underlying application, and so, they are constructed to a particular user's habits in the target application. AHMM and AHMEM allow a wider range of behavior to be modeled but at the cost of having to build the model and set the parameters manually.

Regarding the adaptation to a particular user of the application, from the consistency approaches examined, only Lesh and Etzioni's plan recognizer consider the adaptation to a particular user of the system. They provide an algorithm (called ADAPT) to automatically learn both background goals and predicates that made up goals. Different probabilistic approaches have been also studied. Some of them limit to the tuning of the prior and conditional probabilities assigned to the actions and goals in the model. Markov approaches also are able to learn the structure of the model, and Brown's architecture uses a multi-agent approach to correct the user model (although he does not address the way the correction adaptation agents are implemented). However, only Bauer's approach considered learning patterns in the user behavior by taking a machine learning approach.

Although both Bauer and Lesh and Etzioni's approaches consider a set of past user activities examples as training data, they learn different information. Bauer learns statistical information, while Lesh and Etzioni learn a set of background goals that will filter noisy actions. Bauer's approach is limited to recognizers that use probabilistic information, while Lesh and Etzioni's approach can be applied only to recognizers that produce a single goal.

Lesh and Etzioni's experiments showed that an adaptive system could improve performance. However, his experimental data was provided by other planning systems or simulations rather than observations of humans in realistic scenarios. On the other hand, Bauer only demonstrated that his adaptive mechanism improved performance of plan recognition in the domain of electronic mail.

Finally, we looked through the use of user modeling to improve the plan recognition process. Concretely, diverse machine learning techniques had been employed to consider user's preferences when ascribing the user's intention. The problem of plan recognition in these approaches is view as a problem of classification. Some features of the user behavior are used to "classify" which action the user will perform next.

While they are an interesting approach, they can not be used alone in a plan recognizer, as they not serve well to more elaborated user's intentions, and they do not consider any kind of plan structure.

8. Summary

We have discussed a set of issues that a plan recognizer should address, specially when aiming at being used by interface agents. We have to find a way to represent user's plans and goals and to design an algorithm that, using at least this representation and user's actions over the application as input, could be capable of predicting the user's intention as an output. This prediction should take into account the inherent uncertainty of the user's behavior. It is also very desirable for a plan recognizer to automatically adapt to user's peculiarities, considering its preferences and habits when selecting his intentions.

We have presented different approaches to the problem of plan recognition. There are roughly two kinds of representations used in the literature: consistency and probabilistic approaches. Consistency approaches generally use a plan library in the form of an event hierarchy. Regarding probabilistic approaches, many techniques can be applied to face the problem, such as Bayesian Networks, Probabilistic Grammars, and Markov Models.

We have focused also in the problem of adapting the plan recognition task to the peculiarities of a particular user of the application. In this way, machine learning techniques seem to be likely to turn out well as they can learn a compact representation of user's behavior from raw data of user's activity.

References

- Adisorno, L. and D. Sestero: 1996, 'Using dynamic user models in the recognition of the plans of the user'. In: *User Modeling and User-Adapted Interaction*, Vol. 5. pp. 157–190.
- Albrecht, D. W., I. Zukerman, and A. E. Nicholson: 1998, 'Bayesian Models for Keyhole Plan Recognition in an Adventure Game'. *User Modeling and User-Adapted Interaction* **8**(1-2), 5–47.
- Azarewicz, J., G. Fala, and C. Heithecker: 1989, 'Template- Based Multi-Agent Plan Recognition for Tactical Situation Assessment'. In: *Fifth IEEE Conference on Artificial Intelligence Applications*.
- Bauer, M.: 1995, 'A Demster-Shafer Approach to Modeling Agent Preferences for Plan Recognition'. In: *User Modeling and User-Adapted Interaction*, Vol. 5. pp. 317–348.

- Bauer, M.: 1996, 'Acquisition of User Preferences for Plan Recognition'. In: Springer-Wien (ed.): *Proceedings of the Fifth International Conference on User Modeling (UM '96)*. New York, pp. 105–112.
- Breuker, J.: 1990, 'Conceptual Model of Intelligent Help Systems'. In: J. Breuker (ed.): *EUROHELP: Developing Intelligent Help Systems*. Copenhagen: EC, pp. 41–67.
- Brown, S.: 1998, 'A Decision Theoretic Approach for Interface Agent Development'. Ph.D. thesis, Faculty of the Graduate School of Engineering of the Air Force Institute of Technology Air University.
- Bui, H., S. Venkatesh, and G. West: 2002, 'Policy recognition in the abstract hidden markov model'. *Journal of Artificial Intelligence Research* **17**, 451–499.
- Bui, H. H.: 2003, 'A General Model for Online Probabilistic Plan Recognition'. In: G. Gottlob and T. Walsh (eds.): *IJCAI*. pp. 1309–1318, Morgan Kaufmann.
- Buntine, W.: 1991, 'Theory refinement on Bayesian networks'. In: *Proceedings of the seventh conference (1991) on Uncertainty in artificial intelligence*. San Francisco, CA, USA, pp. 52–60, Morgan Kaufmann Publishers Inc.
- Carberry, S.: 1990, *Plan Recognition in Natural Language Dialogue*. Cambridge, MA, USA: MIT Press.
- Carberry, S.: 2001, 'Techniques for Plan Recognition'. *User Model. User-Adapt. Interact* **11**(1-2), 31–48.
- Charniak, E. and R. Goldman: 1989, 'A Semantics for Probabilistic Quantifier-Free First-Order Languages, with Particular Application to Story Understanding'. In: Morgan-Kaufmann (ed.): *IJCAI-89*. p. 1074.
- Charniak, E. and R. Goldman: 1991, 'A Probabilistic Model of Plan Recognition'. In: *National Conference on Artificial Intelligence (AAAI-91)*.
- Charniak, E. and R. P. Goldman: 1993, 'A Bayesian model of plan recognition'. *Artificial Intelligence* **64**(1), 53–79.
- Chen, Z., F. Lin, H. Liu, Y. Liu, W.-Y. Ma, and L. Wenying: 2002, 'User Intention Modeling in Web Applications Using Data Mining'. *World Wide Web* **5**(3), 181–191.
- Cooper, G. F. and E. Herskovits: 1992, 'A Bayesian Method for the Induction of Probabilistic Networks from Data'. *Mach. Learn.* **9**(4), 309–347.
- Davison, B. D. and H. Hirsh: 1998, 'Predicting Sequences of User Actions'. In: *Predicting the Future: AI Approaches to Time-Series Problems*. Madison, WI, pp. 5–12, AAAI Press. Proceedings of AAAI-98/ICML-98 Workshop, published as Technical Report WS-98-07.
- Geib, C.: 2004, 'Assessing the Complexity of Plan Recognition'. In: *Nineteenth National Conference on Artificial Intelligence (AAAI-04)*. San Jose, CA, pp. 507–512.
- Geib, C. and R. Goldman: 2005, 'Partial Observability and Probabilistic Plan/Goal Recognition'. In: *IJCAI-05 workshop on Modeling Others from Observations*. Edinburgh, Scotland.
- Goldman, R., C. Geib, and C. Miller: 1999, 'A New Model of Plan Recognition'. In: *Proceedings of the 15th Annual Conference on Uncertainty in Artificial Intelligence (UAI-99)*. San Francisco, CA, pp. 245–255, Morgan Kaufmann.
- Gorniak, P.: 2000, 'Keyhole State Space Recognition with Applications to User Modeling'. Master's thesis, University of British Columbia.
- Grosz, B. J. and S. Kraus: 1996, 'Collaborative Plans for Complex Group Action'. *Artificial Intelligence* **86**(2), 269–357.
- Grosz, B. J. and S. Kraus: 1999, 'The evolution of SharedPlans'. In: A. Rao and M. Wooldridge (eds.): *Foundations and Theories of Rational Agencies*. pp. 227–262.

- Grosz, B. J. and C. Sidner: 1990, 'Plans for discourse'. *Intentions and Communications* pp. 417–444.
- Horvitz, E., B. J., D. Heckerman, D. Hovel, and K. Rommelse: 1998, 'The Lumiere project: Bayesian user modeling for inferring the goals and needs of software users'. In: *In Proceedings of the Fourteenth Conference on Uncertainty in Artificial Intelligence*. Madison, WI, pp. 256–265.
- Huber, M. and E. Durfee: 1993, 'Observational Uncertainty in Plan Recognition Among Interacting Robots'. *International Joint Conference on AI (IJCAI). Workshop of Dynamically Interacting Robots* pp. 68–75.
- Huber, M. and R. Simpson: 2004, 'Recognizing the plans of screen reader users'. In: *Modeling Other Agents from Observations (MOO 2004). Workshop W3 at the International Joint Conference on Autonomous Agents and Multi-Agent Systems*. Columbia University, NY, USA.
- Huber, M. J., E. H. Durfee, and M. P. Wellman: 1994, 'The Automated Mapping of Plans for Plan Recognition'. In: R. L. de Mantaras and D. Poole (eds.): *Proceedings of the 10th Conference on Uncertainty in Artificial Intelligence*. San Francisco, CA, USA, pp. 344–351, Morgan Kaufmann publishers Inc.: San Mateo, CA, USA.
- Kautz, H.: 1987, 'A Formal Theory of Plan Recognition'. Ph.D. thesis, Dept. of Computer Science, University of Rochester.
- Kautz, H.: 1991, 'A Formal Theory of Plan Recognition and its Implementation'. In: J. F. Allen, H. A. Kautz, R. Pelavin, and J. Tenenber (eds.): *Reasoning About Plans*. San Mateo (CA), USA: Morgan Kaufmann Publishers, pp. 69–125.
- Kautz, H. and J. Allen: 1986, 'Generalized Plan Recognition'. In: *AAAI*. pp. 32–37.
- Koehn, G. M. and J. E. Greer: 1995, 'The peculiarities of Plan recognition for Intelligent Tutoring Systems'. M. Bauer, editor, *Working notes from the IJCAI workshop 'The next generation of plan recognition systems'* pp. 54–59.
- Lesh, N.; Rich, C. S. C.: 1999, 'Using Plan Recognition in Human-Computer Collaboration'. In: *Seventh International Conference on User Modeling*. Banff Canada.
- Lesh, N.: 1998, 'Scalable and Adaptive Goal Recognition'. Ph.D. thesis, University of Washington.
- Lesh, N. and O. Etzioni: 1995, 'A sound and fast goal recognizer'. In: *Fourteenth International Joint Conference on Artificial Intelligence*. San Francisco, CA, pp. 1704–1710, Morgan Kaufmann.
- Maes, P.: 1994, 'Agents that reduce work and information overload'. *Communications of the ACM* **37**(7), 30–40.
- Maragoudakis, M., A. Thanopoulos, and N. Fakotakis: 2003a, 'User Modeling and Plan Recognition under Conditions of Uncertainty'. In: Matousek and Mautner (eds.): *6th International Conference on Text, Speech and Dialogue (TSD 2003)*. Ceske Budejovice, Czech Republic., pp. 372–379, Springer Verlag.
- Maragoudakis, M., A. Thanopoulos, K. Sgarbas, and N. Fakotakis: 2003b, 'Domain knowledge acquisition and plan recognition by probabilistic reasoning'. In: *Seventh International Conference on Knowledge-Based Intelligent Information & Engineering Systems (?ES'2003)*. University of Oxford, United Kingdom.
- Pearl, J.: 1988, *Probabilistic Reasoning in Intelligent Systems*. San Mateo, California.: Morgan Kaufmann Publishers.
- Peter Spirtes, C. G. and R. Scheines: 1993, *Causation, Prediction, and Search*. Springer.

- Pynadath, D. V. and M. P. Wellman: 1998, 'Generalized Queries on Probabilistic Context-Free Grammars.'. *IEEE Trans. Pattern Anal. Mach. Intell.* **20**(1), 65–77.
- Pynadath, D. V. and M. P. Wellman: 2000, 'Probabilistic State-Dependent Grammars for Plan Recognition.'. In: C. Boutilier and M. Goldszmidt (eds.): *UAI*. pp. 507–514, Morgan Kaufmann.
- Quinlan, J. R.: 1986, 'Induction of Decision Trees'. In: *Machine Learning*, Vol. 1. pp. 81–106.
- Rabiner, L. R.: 1990, 'A tutorial on hidden Markov models and selected applications in speech recognition'. pp. 267–296.
- Rich, C. and C. L. Sidner: 1997, 'COLLAGEN: When Agents Collaborate with People'. In: W. L. Johnson and B. Hayes-Roth (eds.): *Proceedings of the First International Conference on Autonomous Agents (Agents'97)*. New York, pp. 284–291, ACM Press.
- Rich, C. and C. L. Sidner: 1998, 'COLLAGEN: A Collaboration Manager for Software Interface Agents.'. *User Model. User-Adapt. Interact.* **8**(3-4), 315–350.
- Rich, C., C. L. Sidner, and N. Leash: 2001, 'COLLAGEN: Applying collaborative discourse theory to human-computer interaction'. *Artificial Intelligence Magazine* **22**(4), 15–25.
- Schmidt, C. F.; Sridharan, N. S. and J. L. Goodson: 1978, 'The Plan Recognition Problem: An Intersection of Psychology and Artificial Intelligence'. *Artificial Intelligence* **11**, 45–82.
- Shafer, G.: 1976, *A Mathematical Theory of Evidence*. Princeton University Press.
- Stanfill, C. and C. Waltz: 1986, 'Toward memory-based reasoning'. In: *Communications of the ACM*. pp. 1213–1228.
- Wærn, A.: 1996, 'Recognizing Human Plans: Issues for Plan Recognition in Human-Computer Interaction.'. Ph.D. thesis, Royal Institute of Technology.
- Yang, Y. and J. O. Pedersen: 1997, 'A Comparative Study on Feature Selection in Text Categorization'. In: *ICML '97: Proceedings of the Fourteenth International Conference on Machine Learning*. San Francisco, CA, USA, pp. 412–420, Morgan Kaufmann Publishers Inc.

Address for Offprints: ISISTAN Research Institute, UNICEN University, Campus Universitario (B7001BBO), Tandil, Buenos Aires, Argentina